



UNIVERSIDADE FEDERAL DA BAHIA - UFBA
INSTITUTO DE MATEMÁTICA - IM
TRABALHO DE CONCLUSÃO DE CURSO - TCC
MONOGRAFIA



METAESTABILIDADE

JOÃO DELGADO

Salvador - Bahia

27 de julho de 2026

METAESTABILIDADE

JOÃO VICTOR FONSECA DELGADO DA SILVA

Orientador: Prof. Dr. Tertuliano Franco
Santos Franco.

Salvador - Bahia
27 de julho de 2026

Agradecimentos

“Mede o que é mensurável e torna mensurável o que não o é”.

Galileu Galilei (1564-1642)

Resumo

Este trabalho é um estudo sobre metaestabilidade, se guiando desde o entendimento de propriedades de probabilidade até os conceitos de metaestabilidade com uma abordagem de potenciais. Foi utilizado simulações de processos como o andar do bêbado em N dimensões e um caminhar metaestável com quatro estados, com o objetivo de ilustrar fenômenos de metaestabilidade. Utilizando modelos que exploram a evolução de sistemas aleatórios ao longo do tempo, destacando propriedades como convergência de estado e permanência prolongada em regiões específicas do espaço de estados. Por meio de visualizações dinâmicas, evidencia-se como a dinâmica local e a estrutura global de um sistema influenciam o comportamento macroscópico, contribuindo para a compreensão de conceitos fundamentais de Cadeias de Markov e Mecânica Estatística.

Palavras-chave: sistemas de partículas, cadeias de Markov.

Abstract

This work is a study on metastability, following from the understanding of probability properties to the concepts of mortality with a protecial approach. Simulations of processes such as the drunken walk in N dimensions and a metastable walk with four states are used, in order to illustrate phenomena of metastablyship. Using models that explore the evolution of random systems over time, properties such as convergence between distinct configurations and prolonged permanence in specific regions of state space are highlighted. Through dynamic visualizations, it is evident how the local dynamics and the global structure of a system influence the macroscopic behavior, contributing to the understanding of fundamental concepts of Markov chains and Static Mechanics.

Keywords: random graphs, phase transition.

Conteúdo

Introdução	1
1 Noções básicas de medida e probabilidade	2
1.1 Cadeia de Markov	6
1.2 Função de representação aleatória	7
1.3 Passeio aleatório em grafos	9
1.4 Distribuição estacionária	10
2 Metaestabilidade para o passeio aleatório no hipercubo	11
2.1 Tipos de passeios aleatórios homogêneos:	11
2.1.1 Passeio homogêneo preguiçoso $\sigma(t)$	11
2.1.2 Passeio homogêneo não preguiçoso $\xi(t)$	12
3 Abordagem Potencial para Metaestabilidade	19
3.0.1 Caminhar aleatório em malhas	19
3.1 Simulações em python	21
3.1.1 Caminhar aleatório com acoplamento	22
3.1.2 Metaestabilidade: Uma abordagem com potencial	26
3.1.3 Convergência de distribuição: Tempo de saída	36
Referências Bibliográficas	40
Índice Remissivo	41

Lista de Figuras

3.1	Malha $\Lambda_N = \{0, \dots, N\}^d$ com um elemento de E_N destacado por azul.	19
3.2	Um caminhar aleatório com dois estados	20
3.3	Acoplamento de dois passeios aleatórios	22
3.4	Andar mestaestável com 4 estados	26
3.5	Convergência da distribuição do tempo de fuga de um estado	36

Introdução

A metaestabilidade é um fenômeno que surge em diversos contextos, especialmente no estudo de sistemas dinâmicos e probabilísticos. Sua investigação ganhou notoriedade a partir da necessidade de compreender comportamentos aparentemente estáveis em sistemas que, a longo prazo, sofrem transições abruptas entre estados. Historicamente, esse tema está associado ao desenvolvimento da teoria das probabilidade, da Mecânica Estatística e das cadeias de Markov, áreas impulsionadas por problemas oriundos tanto da Física quanto da matemática pura. Contribuições fundamentais de diversos pesquisadores ao longo do século XX permitiram a consolidação dessas ideias, estabelecendo as bases teóricas para o estudo rigoroso da metaestabilidade.

De forma geral, a metaestabilidade descreve situações em que um sistema permanece por longos períodos em estados que não são verdadeiramente estáveis, mas que apresentam uma estabilidade aparente antes de ocorrer uma transição para um estado mais estável. Esse comportamento é particularmente relevante em modelos probabilísticos, nos quais a análise do tempo de permanência e das transições entre estados desempenha um papel central. Assim, o estudo da metaestabilidade torna-se essencial para a compreensão de fenômenos complexos, com aplicações que vão desde processos físicos até algoritmos estocásticos.

Neste trabalho, serão apresentados inicialmente conceitos fundamentais necessários para o desenvolvimento do tema, com destaque para noções básicas de probabilidade e cadeias de Markov. Em seguida, será abordado um modelo específico de sistema em tempo discreto definido em um reticulado (*lattice*), com base em uma aula ministrada pelo professor Peixoto em um colóquio do IMPA, que servirá como motivação e exemplo concreto do fenômeno estudado. Por fim, será explorada uma abordagem baseada em potenciais, fundamentada no artigo de Claudio Landim, permitindo uma análise mais aprofundada das propriedades metaestáveis e de suas implicações teóricas.

Capítulo 1

Noções básicas de medida e probabilidade

Nesse capítulo vamos falar das noções iniciais de probabilidade para que possamos compreender os processos de metaestabilidade. Inicialmente falaremos da construção da σ -álgebra e as consequências de imbuir esse objeto com uma medida, assim seguindo para a construção da medida de probabilidade. Em sequência, falamos um sobre as propriedades das cadeias de Markov, que são um dos objetos mais importantes quando falamos de metaestabilidade.

Definição 1.1. *Uma álgebra de um conjunto Ω é uma classe $\mathcal{F}$ de subconjuntos de Ω tal que:*

- i. $\Omega \in \mathcal{F}$,*
- ii. $A \in \mathcal{F}$ implica que $A^c \in \mathcal{F}$,*
- iii. $A, B \in \mathcal{F}$ implica que $A \cup B \in \mathcal{F}$.*

Uma álgebra é uma classe que se pode realizar operações elementares de conjuntos, o que é característica muito boa, pois como funções trabalham com conjunto e, sem esses objetos, ficamos muito restritos ou até de mãos atadas ao desenvolver uma teoria. Contudo, a álgebra ainda não é uma estrutura adequada para trabalhar com probabilidade, pois, apesar das uniões de dois elementos da álgebra estarem na álgebra, a união infinitesimal desses elementos pode não estar no conjunto. Logo, precisamos de uma estrutura que seja fechada para uniões enumeráveis.

Definição 1.2. *Uma σ -álgebra é uma álgebra fechada por uniões enumeráveis, ou seja,*

- iv. $A_j \in \mathcal{F}$ tal que $j = 1, 2, \dots$ implica que $\bigcup_{j=1}^{\infty} A_j \in \mathcal{F}$.*

Exemplo 1.1. Um exemplo básico de uma σ -álgebra é o que corresponde ao lançamento de um dado de 6 lados honesto. O nosso conjunto de estudo seria os lados do dado, $D = \{1, 2, 3, 4, 5, 6\}$ e nossa σ -álgebra é $\mathcal{S} = \mathcal{P}(D)$, as partes de D .

Definição 1.3 (σ -álgebra gerada por uma classe de subconjuntos). Seja $\mathcal{E}$ uma classe de subconjuntos de Ω . Então definamos $\sigma(\mathcal{E})$ como a única σ -álgebra gerada por $\mathcal{E}$ que satisfaz:

- i. $\sigma(\mathcal{E})$ é σ -álgebra,
- ii. $\mathcal{E} \subseteq \sigma(\mathcal{E})$,
- iii. Se $\mathcal{S}$ é σ -álgebra $\mathcal{E} \subseteq \mathcal{S}$, então implica que $\sigma(\mathcal{E}) \subseteq \mathcal{S}$.

Exemplo 1.2. Ainda pensando no dado, tomemos uma classe de conjuntos $\mathcal{E} = \{\{1, 2\}\}$. Podemos criar uma σ -álgebra, $\sigma(\mathcal{E})$, adicionando $D, \{1, 2\}, \emptyset, \{3, 4, 5, 6\}$ em $\sigma(\mathcal{E})$, logo temos $\sigma(\mathcal{E}) = \{D, \emptyset, \{1, 2\}, \{3, 4, 5, 6\}\}$. Perceba que as uniões dois-a-dois são fechadas, então $\sigma(\mathcal{E})$ é σ -álgebra.

Criado o nosso espaço onde vamos estudar (espaços mensuráveis), podemos começar a elaborar a ideia de probabilidade.

Definição 1.4. Uma medida em um espaço mensurável, $(\Omega, \mathcal{S})$, é uma função $\mu : \mathcal{S} \rightarrow [0, +\infty]$ que satisfaz:

- i. $\mu(\emptyset) = 0$,
- ii. $\mu\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mu(A_i)$ para todo $A_i \in \mathcal{S}$ dois a dois disjuntos.

A tripla $(\Omega, \mathcal{S}, \mu)$ denomina-se espaço de medida.

Observe que isso inclui somas finitas, pois podemos considerar os A_n tal qual para todo $n > n_0$ é igual a $\emptyset$. Um exemplo importante de medida é a delta de Dirac:

Exemplo 1.3. Seja Ω um conjunto e consideremos $\mathcal{P}(\Omega)$, as partes de Ω , então dado um $p \in \Omega$, seja a função $\delta_p : \mathcal{P}(\Omega) \rightarrow [0, +\infty]$:

$$\delta_p(A) = \begin{cases} 1, & \text{se } p \in A; \\ 0, & \text{c.c.} \end{cases}$$

Vamos agora ver algumas propriedades das medidas que não são nenhuma surpresa, mas que precisamos formular.

Teorema 1.1. Seja $(\Omega, \mathcal{S}, \mu)$ um espaço de medida e $A, B, A_1, A_2, \dots \in \mathcal{S}$.

1. $\mu(A^c) = \mu(\Omega) - \mu(A)$,
2. Se $A \subseteq B$, então vale que $\mu(A) \leq \mu(B)$ e $\mu(B \setminus A) = \mu(B) - \mu(A)$,
3. $\mu(A \cup B) = \mu(A) + \mu(B) - \mu(A \cap B)$,
4. $\mu\left(\bigcup_{n=1}^{\infty} (A_n)\right) \leq \sum_{n=1}^{\infty} \mu(A_n)$,
5. Se $\mu(A_n) < \infty$, $A_n \uparrow A$ ou $A_n \downarrow A$, então $\mu(A_n) \rightarrow \mu(A)$.

Demonstração. 1. Como $\Omega = A \cup A^c$, temos que $\mu(\Omega) = \mu(A \cup A^c) = \mu(A) + \mu(A^c)$.

2. Como $A \subseteq B$, então $B = A \cup B \setminus A \Rightarrow \mu(B) = \mu(A \cup B \setminus A) = \mu(A) + \mu(B \setminus A) \Rightarrow \mu(B \setminus A) = \mu(B) - \mu(A)$.
3. Usando o item (ii) temos que $\mu(A \cup B) = \mu(A) + \mu(B \setminus A)$. Como $\mu(B) = \mu(B \setminus A) + \mu(A \cap B)$, então $\mu(A \cup B) = \mu(A) + \mu(B) - \mu(A \cap B)$.
4. A partir de uma sequência de A_n podemos criar uma sequência disjunta. Seja $B_1 = A_1$, $B_2 = A_2 \setminus A_1$, $B_3 = A_3 \setminus (A_2 \cup A_1)$, etc. Ou seja, $B_n = A_n \setminus (\bigcup_{j=1}^{n-1} A_j)$, perceba que B_n é uma sequência de conjuntos disjunta. Assim $\mu\left(\bigcup_{n=1}^{\infty} A_n\right) = \mu\left(\bigcup_{n=1}^{\infty} B_n\right) = \sum_{n=1}^{\infty} \mu(B_n) \leq \sum_{n=1}^{\infty} \mu(A_n)$.
5. Suponhamos que $A_n \uparrow A$, então $A_n \subset A_{n+1}$. Como é uma sequência de conjuntos arbitrário, a fim de obter melhor controle, podemos criar uma sequência B_n de conjuntos disjunta tal que $\bigcup_n^{\infty} B_n = \bigcup_n^{\infty} A_n$, definindo $B_n = A_n \setminus A_{n-1}$ sendo $A_0 = \emptyset$.
 Fixe um $k \in \mathbb{N}$ e $i \geq 1$. Daí, $\bigcup_i^k B_i = \bigcup_i^k (A_i \setminus A_{i-1})$ nos leva a

$$\begin{aligned}
 \mu\left(\bigcup_i^k B_i\right) &= \mu\left(\bigcup_i^k (A_i \setminus A_{i-1})\right), \\
 &= \sum_i^k \mu(A_i \setminus A_{i-1}), \\
 &= \sum_i^k (\mu(A_i) - \mu(A_{i-1})).
 \end{aligned}$$

Como o lado direito da equação é uma soma telescópica, temos que $\mu(\bigcup_i^k B_i) = \mu(A_k) - \mu(A_0)$. Logo,

$$\begin{aligned} \lim_{k \rightarrow \infty} \mu \left(\bigcup_i^k B_i \right) &= \lim_{k \rightarrow \infty} \mu(A_k), \\ \Rightarrow \mu \left(\bigcup_i^\infty B_i \right) &= \mu \left(\bigcup_i^\infty A_i \right) = \mu(A). \end{aligned}$$

Para provar $A_n \downarrow A$, podemos utilizar o que acabamos de provar. Tome a sequência $A_n^c \uparrow A^c$ e lembrando que $P(A_n) = 1 - P(A_n^c)$, então se $n \rightarrow \infty$ temos que $\lim_{n \rightarrow \infty} P(A_n) = 1 - P(A^c)$. □

Agora que entendemos um pouco sobre medidas e σ -álgebras, podemos começar a adentrar no assunto de probabilidade. Contudo o que seria uma probabilidade? É intuitivo pensar que probabilidade satisfaça as definição de medida, pois com esse objeto temos o intuito de medir quão provável um evento pode ocorrer.

Definição 1.5. *Seja $(\Omega, \mathcal{S}, \mu)$ um espaço de medida. Se $\mu : \mathcal{S} \rightarrow \mathbb{R}$ é uma função que satisfaz:*

- i. $\mu(\Omega) = 1$,
- ii. $\mu \left(\bigcup_{i=1}^\infty A_i \right) = \sum_{i=1}^\infty \mu(A_i)$ para todo $A_i \in \mathcal{S}$ dois a dois disjuntos.

Então μ é dita uma medida de probabilidade. Uma tripla $(\Omega, \mathcal{S}, \mu)$ denomina-se espaço de probabilidade.

Perceba que pela afirmação 1, temos que $\mu(\Omega) = 1 - \mu(\emptyset) = 1$. Logo, essa definição para probabilidade μ , nos diz que também é medida, assim tudo que vimos para medida vale para probabilidade.

Seja $(\Omega, \mathcal{S}, \mu)$ um espaço de probabilidade. Tome dois eventos $A, B \in \mathcal{S}$ tais que $\mu(A) = a$ e $\mu(B) = b$. Qual é a probabilidade do evento acontecer A se sabemos que o evento B acontece?

Definição 1.6. *Seja $(\Omega, \mathcal{S}, \mu)$ um espaço de probabilidade e $A, B \in \mathcal{S}$ tal que $\mu(B) > 0$. Uma probabilidade condicional de A dado B é $\mathbb{P} : \mathcal{S} \rightarrow \mathbb{R}$*

$$\mathbb{P}(A|B) = \frac{\mu(A \cap B)}{\mu(B)}.$$

Quando $\mu(B) = 0$, então definimos $\mathbb{P}(A|B) = \mathbb{P}(A)$.

A probabilidade condicional diferente da noção usual de probabilidade, contextualiza as chances de um evento acontecer. Perceba que quando $\mathbb{P}(B) > 0$, a primeira nada mais é do que relativização da probabilidade do evento A para o evento B . Veja que estamos falando de eventos, elementos da σ -álgebra e não qualquer subconjunto de Ω .

Teorema 1.2 (Regra do produto). *Em $(\Omega, \mathcal{S}, \mu)$. Dados os eventos $A_1, A_2, \dots, A_n \in \mathcal{S}$,*

$$\mathbb{P}\left(\bigcap_{i=1}^n A_i\right) = \mathbb{P}(A_1)\mathbb{P}(A_2|A_1)\mathbb{P}(A_3|A_1 \cap A_2) \dots \mathbb{P}\left(A_n \left| \bigcap_{i=1}^{n-1} A_i\right.\right).$$

Definição 1.7 (Eventos independentes). *Seja $A, B \in \mathcal{S}$ são independentes se*

$$\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B).$$

Logo, se a sequência de eventos A_i são independentes, então $\mathbb{P}\left(\bigcap_{i=1}^n A_i\right) = \prod_{i=1}^n \mathbb{P}(A_i)$, pois $\mathbb{P}(A_j|A_i) = A_j$.

1.1 Cadeia de Markov

Definição 1.8 (Cadeia de Markov finita). *Seja $(\Omega, \sigma\{\Omega\}, \mathbb{P})$ um espaço de probabilidade e $(X_1, X_2, X_3, \dots)$ uma sequência aleatória de variáveis que satisfaz, juntamente com uma matriz de transição $\mathbb{P}$, a propriedade de Markov, definida da seguinte forma: Se para $\forall x, y \in \Omega$, para todo $t \geq 1$, e para todo evento $H_{t-1} = \bigcap_{s=0}^{t-1} \{X_s = x_s\}$ satisfaz $\mathbb{P}(H_{t-1} \cap \{X_t = x\}) > 0$, nos temos que*

$$\mathbb{P}\left\{X_{t+1} = y | H_{t-1} \cap \{X_t = x\}\right\} = \mathbb{P}\{X_{t+1} = y | \{X_t = x\}\}.$$

Em termos mais amigáveis, a probabilidade de a sequência chegar a um determinado estado depende apenas do estado anterior, podemos pensar nisso como uma sequência que apresenta perda de memória. Como estamos trabalhando com probabilidade, a soma das chances de transição de um estado para todos os outros deve ser igual a 1. Por isso, a nossa matriz de transição precisa ser estocástica.

Definição 1.9 (Matriz estocástica). *Denominamos P de matriz estocástica, se as suas entradas são não-negativas e a soma de elementos de cada coluna da matriz é igual a um, isso é, $\sum_{j \in |\Omega|} P_{ij} = \mathbb{P}(x, \cdot) = 1 \forall x \in \Omega$.*

Em muitos casos gostaríamos de trabalhar com distribuições de pontos diferentes do espaço, $\mathbb{P}(x, \cdot), \mathbb{P}(y, \cdot)$, então para facilitar a compreensão, vamos utilizar a notação $\mathbb{E}_x, \mathbb{P}_x$.

$$\mathbb{E}_x(f(X_1)) = \sum_y f(y) \mathbb{P}_x(X_1 = y) = \sum_y f(y) \mathbb{P}(x, y) = \mathbb{P}f(X).$$

tal que f é uma função. Notação: $\mathbb{P}_x(X_t = y) = (\delta_x \mathbb{P}^t)(y) = \mathbb{P}^t(x, y)$.

1.2 Função de representação aleatória

Definição 1.10 (Função de representação aleatória de uma matriz P em um espaço de estados Ω). *A função f é uma representação aleatória se $f : \Omega \times \Lambda \rightarrow \Lambda$, onde Λ são os valores da variável aleatória Z definida em Ω , satisfaz*

$$\mathbb{P}(f(x, Z(x)) = Y) = \mathbb{P}(x, y).$$

A função de representação é uma ótima ferramenta para substituírmos a matriz de transição quando temos uma quantidade extensa de estados tornando as operações laboriosas. Seja uma sequência de variáveis $(Z_n)_n$ iid's com mesma distribuição de Z e X_0 com distribuição μ , podemos definir uma sequência de variáveis aleatórias $(X_n)_n$ como

$$X_n = f(X_{n-1}, Z_n) \quad n \geq 1,$$

tal que essa sequência é uma cadeia de Markov com uma matriz de transição P com distribuição μ .

Proposição 1.1. *Toda matriz de transição em um espaço de Markov finito tem uma função de representação aleatória.*

Demonstração. Seja P uma matriz de transição de um espaço de Markov com o espaço de estados $\Omega = \{x_1, x_2, \dots, x_n\}$ e seja $\Lambda = [0, 1]$, onde as variáveis aleatórias auxiliares $Z, Z_1, Z_2, \dots$ são iid's nesse intervalo. Definamos $F_{ij} = \sum_{k=1}^j \mathbb{P}(x_i, x(k))$, então podemos definir f como $f(x_i, Z) := x_j$ quando $F_{i,j-1} < Z < F_{i,j}$. Portanto,

$$\mathbb{P}\{f(x_i, Z) = x_j\} = \mathbb{P}\{F_{i,j-1} < Z < F_{i,j}\} = \mathbb{P}(x_i, x_j).$$

□

Veremos em seguidas algumas propriedades dessas cadeias de Markov que nos permite extrair algumas informações.

Definição 1.11 (Irredutível). *Seja P uma cadeia, se para todo $x, y \in \Omega$, existe um inteiro t tal que $\mathbb{P}^t(x, y) > 0$.*

Definição 1.12 (Período de x). *Definamos o conjunto $T(x) = \{t \geq 1 : \mathbb{P}^t(x, x) > 0\}$. O período de x é o maior divisor comum (mdc de $T(x)$).*

Proposição 1.2. *Se P é irredutível, então $\text{mdc}(T(x)) = \text{mdc}(T(y))$ para $x, y \in \Omega$.*

Demonstração. Seja $k \in T(x)$. Sabemos que existe r e l tal que $\mathbb{P}^r(x, y) > 0$ e $\mathbb{P}^l(x, y) > 0$. Seja $m = r + l$, é direto ver que $m \in T(x) \cap T(y)$. Provaremos que $k + m \in T(y)$,

$$\begin{aligned}\mathbb{P}^{k+m}(x, x) &= \mathbb{P}^k(x, x)\mathbb{P}^r(x, y)\mathbb{P}^l(y, x) \\ &= \mathbb{P}^l(y, x)\mathbb{P}^k(x, x)\mathbb{P}^r(x, y) \\ &= \mathbb{P}^{l+k+r}(y, y) = \mathbb{P}^{m+k}(y, y).\end{aligned}$$

Então $k + m \in T(y)$, então $k \in \{n - m : n \in T(y)\} \Rightarrow T(x) \subset T(y) - m$. Portanto, como $m \in T(y)$, $\text{mdc } T(y)$ divide todo elemento de $T(x)$, consequentemente $\text{mdc}(T(y)) \leq \text{mdc}(T(x))$. Para $\text{mdc}(T(x)) \leq \text{mdc}(T(y))$ é análogo. Portanto, $\text{mdc}(T(y)) = \text{mdc}(T(x))$. $\square$

Definição 1.13 (Aperiódico). *Se em uma cadeia o $\text{mdc}(T(x)) \forall x \in \Omega$ é 1, então denominamos aperiódico. Se a cadeia não é aperiódico, então é periódico.*

Proposição 1.3. *Se P é irredutível, então existe um inteiro r tal que $\mathbb{P}^r(x, y) > 0$ para $\forall x, y \in \Omega$.*

Demonstração. Antes de provar essa proposição, iremos demonstrar um lema que nos garantirá que a partir de um inteiro vai ser possível o retorno de um $x \in \Omega$.

Lema 1.1. *Seja $S \subset \mathbb{Z}^+$ tal qual é fechado por adição e de $\text{mdc}(S) = 1$, então existe $m_s \in \mathbb{Z}^+$ tal que para todos $m \geq m_s$ pode ser escrito como combinação linear de elementos de S .*

Demonstração. Como o $\text{mdc}(S) = 1$, podemos aplicar o Teorema de Bézout. Seja $a, b \in S, \exists x, y \in \mathbb{Z}, ax + by = 1$. O caso de $x, y \geq 0$, então está feito. O caso de $x, y < 0$, não ocorre, pois $a, b \leq 0$. Então só nos resta o caso de $x > 0$ e $y < 0$ ou contrário. Suponhamos que $y > 0$ e $x < 0$, e seja $k \in \mathbb{Z}^+$, logo $\exists r, q \in \mathbb{Z}^+ (k = aq + r \wedge a \geq r \geq q)$.

$$\begin{aligned}1 &= xa + yb \\ r &= rxa + ryb, \\ k &= aq + r = rxa + aq + ryb \\ k &= (rx + q)a + ryb.\end{aligned}$$

Como $y > 0$ e $r \leq 0$, então só precisamos verificar $xr + q$. Lembrando que $x < 0$, logo $(-x) > 0$.

$$xr + q > 0 \Leftrightarrow q > (-x)r.$$

Como $a > r$, então $q > (-x)a$. □

Agora, podemos provar a Proposição 1.3. Vamos provar que $\forall x \in \Omega$, $T(x)$ é fechado por adição. Seja $a, b \in T(x)$, $\mathbb{P}^{a+b}(x, x) = \mathbb{P}^a(x, x)\mathbb{P}^b(x, x) > 0$, então $a + b \in T(x)$. Logo, pelo lema anterior, sabemos que existe t_x tal que $t \geq t_x$, $\mathbb{P}^{t_x}(x, x) > 0$. Como sistema é irredutível, então existe um inteiro $k = k(x, y)$ tal que $\mathbb{P}^k(x, y) > 0$. Logo, fixado x , seja $t = t_x + r(x, y)$,

$$\mathbb{P}^t(x, y) \geq \mathbb{P}^{t_x}(x, x)\mathbb{P}^{r(x, y)}(x, y) > 0.$$

□

1.3 Passeio aleatório em grafos

Um grafo $G = (V, E)$ consiste em um conjunto de vértice V e um conjunto de aresta E , onde $E \subset \{\{x, y\} : x, y \in V \text{ e } x \neq y\}$ e V é o conjunto de pontos tal que para cada par de ponto ligados por um linha. Denotando $\{x, y\} \in E$ como $x \sim y$, chamamos y de vizinhança de x . E o total de vizinhança de x como o grau de x , ou seja, $|\{x \sim y : y \in V\}| = \deg(x)$.

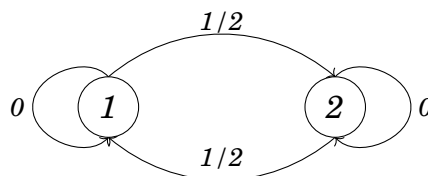
Como um exemplo inicial, podemos definir um grafo e criar uma função de probabilidade a partir da vizinhança de um ponto, tornando a escolha de um vértice pular para outro, ser uniforme.

$$\mathbb{P}(x, y) = \begin{cases} \frac{1}{\deg(x)}; & \text{se } x \sim y, \\ 0; & \text{c.c.} \end{cases}$$

Exemplo 1.4. Seja G um grafo de 2 vértices. Então temos uma matriz de transição

$$P = \begin{pmatrix} \frac{1}{2} & 1 \\ \frac{1}{2} & 0 \end{pmatrix}.$$

Ou seja,



1.4 Distribuição estacionária

Em caminhadas aleatórias podemos andar tanto e acabar tendendo a algum lugar. Assim, se verificarmos que a distribuição de uma caminhada aleatória está tendendo para algum lugar, então dizemos que ela é estacionária.

Definição 1.14. *Seja P uma matriz de transição em grafo G . Definimos como uma distribuição estacionária, quando uma medida de probabilidade π satisfaz*

$$\pi = \pi P.$$

De forma equivalente podemos reescrever nossa definição como

$$\pi(y) = \sum_{x \in V} \pi(x) P(x, y).$$

Exemplo 1.5. *Utilizando o simples andar aleatório sobre um grafo $G = (V, E)$. Para qualquer $y \in V$*

$$\sum_{x \in V} \deg(x) P(x, y) = \sum_{x \sim y} \frac{\deg(x)}{\deg(x)} = \deg(y)$$

Utilizando como normalizador $\sum_{y \in V} \deg(y) = 2|E|$, podemos dizer que a medida de probabilidade

$$\pi(y) = \frac{\deg(y)}{2|E|} \quad \forall y \in V$$

é uma distribuição estacionária.

Teorema 1.3. *Seja x e y dois pontos de distância $k \geq 1$ no toro $\mathbb{Z}_N$ e seja τ_y o primeiro tempo de chegada em y . Então existe uma constante $0 < c_d \leq C_d < \infty$ tal que*

$$\begin{aligned} c_d N^d &\leq \mathbf{E}_x(\tau_y) \leq C_d N^d && \text{unifomemente em } k \text{ se } d \geq 3 \\ c_2 N^2 \log(k) &\leq \mathbf{E}_x(\tau_y) \leq C_2 N^2 \log(k+1) && \text{se } d = 2 \end{aligned}$$

Capítulo 2

Metaestabilidade para o passeio aleatório no hipercubo

Chamaremos $H_N = \{-1, +1\}^N$ um conjunto de configurações de N spins (as entradas) com possíveis valores $\{+1, -1\}$. Os elementos do H_N são representados por ν e ζ , ou seja, $\nu = (\nu_1, \dots, \nu_N)$ e $\zeta = (\zeta_1, \dots, \zeta_N)$ onde ζ_i e $\nu_i \in \{+1, -1\}, \forall i \in \{1, \dots, N\}$. Seja $\nu \in H_N$, o valor da troca no spin j de ν é denotado por ν^j .

$$(\nu^j)_i = \begin{cases} \nu_i & , \text{se } i \neq j; \\ -\nu_j & , \text{se } i = j. \end{cases}$$

2.1 Tipos de passeios aleatórios homogêneos:

2.1.1 Passeio homogêneo preguiçoso $\sigma(t)$

Seja $\nu \in H_N$ e $\sigma(0) = \nu$, $\sigma(t)$ é uma evolução estocástica em relação a um tempo discreto, onde para cada t escolhemos $i \in \{1, \dots, N\}$ com probabilidade $\frac{1}{N}$ e escolheremos em seguida o spin $\sigma_i = +1$ com probabilidade $\frac{1}{2}$. Podemos interpretar o passeio homogêneo como a sequência de duas variáveis aleatórias $I(t)$ e $U(t)$, definidas em $(\Omega_N, \mathcal{F}_N, \mathbb{P}_N)$, sendo $I(t) \in \{1, \dots, N\}$ independente e $\forall k \in \{1, \dots, N\}; \mathbb{P}_N(I(t) = k) = \frac{1}{N}$ e $U(t) \in [0, 1]$ independente e $\forall u \in [0, 1]; \mathbb{P}_N(U(t) < u) = u$, ou seja, identicamente distribuída uniformemente em $[0, 1]$.

Assim,

$$(\sigma_j(t)) = \begin{cases} \sigma(t-1) & , \text{se } I(t) \neq j; \\ +1 & , \text{se } I(t) = j; U(t) < \frac{1}{2}; \\ -1 & , \text{se } I(t) = j; U(t) \geq \frac{1}{2}. \end{cases}$$

2.1.2 Passeio homogêneo não preguiçoso $\xi(t)$

Seja $\xi(0) = \nu \in H_N$ e $\xi(t)$ um passeio aleatório no H_N definido em $(\Omega_0, \mathcal{F}_0, \mathbb{P}_0)$ com probabilidade de transição $\mathbb{P}(\xi(t+1) = \nu^i | \xi(t) = \nu) = \frac{1}{N}$ para todo $k \geq 1$, $i \in \{1, \dots, N\}$. Perceba que $\mathbb{P}(\xi(t+1) = \nu | \xi(t) = \nu) = 0$, pois $\sum_i \mathbb{P}(\xi(t+1) = \nu^i | \xi(t) = \nu) = \left(N \cdot \frac{1}{N}\right) = 1$, ou seja, a probabilidade dele mudar alguma entrada/spin 100%.

Agora vamos introduzir algumas notações básicas para não ocorrer nenhum tipo de ambiguidade.

Notações

σ^- é o passeio homogêneo com configuração inicial $\sigma^-(0) = (-1, \dots, -1)$.

σ^+ é o passeio homogêneo com configuração inicial $\sigma^+(0) = (+1, \dots, +1)$.

σ^ν é o passeio homogêneo com configuração inicial $\sigma^\nu(0) = \nu, \nu \in H_N$.

ξ^- é o passeio homogêneo com configuração inicial $\xi^-(0) = (-1, \dots, -1)$.

ξ^+ é o passeio homogêneo com configuração inicial $\xi^+(0) = (+1, \dots, +1)$.

ξ^ν é o passeio homogêneo com configuração inicial $\xi^\nu(0) = \nu; \nu \in H_N$.

$t_N^- := \inf\{t > 0 : \sigma^+(t) = \sigma^-(t)\}$: Tempo de parada/encontro.

$t_N^{\nu, \zeta} := \inf\{t > 0 : \sigma^\nu(t) = \sigma^\zeta(t)\}$: Tempo de parada/encontro.

$D_N(t) := \left| \frac{K^+(t) - P^-(t)}{2N} \right|$ tal que K, P é um passeio preguiçoso ou não preguiçoso:

É a distância no tempo t .

$D_N^{\nu, \zeta}(t) := \left| \frac{K^\nu(t) - P^\zeta(t)}{2N} \right|$ tal que K, P é um passeio preguiçoso ou não preguiçoso:

É a distância no tempo t .

$V[0, N^\gamma] := \{\sigma^+(0), \dots, \sigma^+(N^\gamma)\}$.

O Teorema 2.1 nos dá uma estimativa do crescimento do tempo que leva para dois pontos do hipercubo se encontrarem quando $N \rightarrow \infty$.

Teorema 2.1. *Seja $t_N^- = \inf(t > 0 : \sigma^+(t) = \sigma^-(t))$. Então,*

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\left| \frac{t_N^-}{N \log N} - 1 \right| > \varepsilon \right) = 0, \forall \varepsilon > 0.$$

Demonstração. Para provar o teorema é suficiente mostrar que

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\frac{t_N^-}{N \log N} \right) = 1.$$

Seja $D_N(t) = \frac{1}{2N} \sum_{i=1}^N |\sigma_i^+(t) - \sigma_i^-(t)|$ a distância no instante n entre σ^+ e σ^- .

Pela evolução de $\sigma(t)$ temos que $D_N(t)$ é uma cadeia de Markov em $\{0, \frac{1}{N}, \dots, \frac{N-1}{N}, 1\}$ sendo a probabilidade de transição dada por:

$$\mathbb{P}_{xy} = \begin{cases} 1-x & , \text{ se } y = x, \\ x & , \text{ se } y = x - \frac{1}{N}, \\ 0 & , \text{ c.c.} \end{cases}$$

Definimos $t_N^- = \sum_{k=1}^N \lambda_k$, onde $\lambda_1 = 1$ e λ_k são variáveis aleatórias de geométricas de parâmetros $p_k = 1 - \frac{k-1}{N}$. λ_k são o menor tempo que o processo $D_N(t)$ demora para sair de $1 - \frac{k-1}{N}$ para $1 - \frac{k}{N}$.

Vamos ver a esperança:

$$\begin{aligned} \mathbb{E}(\lambda_k) &= \sum_n^{\infty} \mathbb{P}(\lambda_k = n) \cdot n, \\ &= 1 \cdot p_k + 2 \cdot p_k(1 - p_k) + 3 \cdot p_k(1 - p_k)^2 + \dots \end{aligned}$$

Como $\mathbb{E}(\lambda_k) \cdot (1 - p_k) = 1 \cdot p_k(1 - p_k) + 2 \cdot p_k(1 - p_k)^2 + 3 \cdot p_k(1 - p_k)^3 + \dots$

$$\begin{aligned} \mathbb{E}(\lambda_k) - (1 - p_k) \cdot \mathbb{E}(\lambda_k) &= p_k + p_k \cdot (1 - p_k) + p_k \cdot (1 - p_k)^2 + \dots, \\ p_k \mathbb{E}(\lambda_k) &= p_k(1 + (1 - p_k) + (1 - p_k)^2 + \dots), \\ \mathbb{E}(\lambda_k) &= (1 + (1 - p_k) + (1 - p_k)^2 + \dots), \\ &= \frac{1}{1 - (1 - p_k)} = \frac{1}{p_k} = \frac{N}{N + 1 - k}. \end{aligned}$$

Agora vamos observar a variância:

$$\text{Var}(\lambda_k) = \mathbb{E}(\lambda_k - \mathbb{E}(\lambda_k))^2 = \mathbb{E}(\lambda_k^2) - \mathbb{E}(\lambda_k)^2.$$

Vamos obter $\mathbb{E}(\lambda^2)$:

$$\mathbb{E}(\lambda_k^2) = \sum_x^{\infty} x^2 \cdot \mathbb{P}(\lambda_k = x) = \sum_x^{\infty} x^2 \cdot (1 - p_k) p_k^{x-1},$$

$$= \sum_x^\infty x^2 q^{x-1} p_k = p_k \sum_x^\infty x \frac{d}{dq_k} q_k^x.$$

Sendo $q_k = (1 - p_k)$.

$$\begin{aligned} \mathbb{E}(\lambda_k^2) &= p_k \frac{d}{dq_k} \cdot q \sum_x^\infty x \cdot q_k^{x-1} = p_k \frac{d}{dq_k} \cdot q \sum_x^\infty \frac{d}{dq_k} q_k^x, \\ &= p_k \frac{d}{dq_k} \cdot q \cdot \frac{d}{dq_k} \sum_x^\infty q_k^x = p_k \frac{d}{dq_k} \cdot q \cdot \frac{d}{dq_k} \cdot \left(\frac{q_k}{1 - q_k} \right), \\ &= p_k \left(\frac{1 + q_k}{(1 - q_k)^2} \right) = \frac{2 - p_k}{p_k^2}. \end{aligned}$$

Assim, $\text{Var}(\lambda_k) = \frac{2-p_k}{p_k^2} - \frac{1}{p_k} = \frac{1-p_k}{p_k^2}$. Substituindo $p_k = 1 - \frac{k-1}{N}$, temos $\text{Var} = \frac{N(k-1)}{(N+1-k)}$.

Logo,

$$\begin{aligned} \mathbb{E}(t_N^-) &= \mathbb{E} \left(\sum_{k=0}^N \lambda_k \right) = \sum_{k=0}^N \frac{N}{N+1-k}, \\ &= N \sum_{k=0}^N \frac{1}{N+1-k} = N \sum_{k=1}^N \frac{1}{k}. \end{aligned}$$

Como $\int_1^N \frac{1}{x} dx = \log N$, então $N \log(N-1) \leq \mathbb{E}(\lambda_k) = N \sum_{k=1}^N \frac{1}{k} \leq N \log(N)$.

Quando $n \rightarrow \infty$, $\mathbb{E}(t_N^-) \sim N \log(N)$. Logo, pela desigualdade de Tchebyshev:

$$\begin{aligned} \mathbb{P}(t_N^- - \mathbb{E}(t_N^-) | \varepsilon \cdot \mathbb{E}(t_N^-)) &\leq \frac{\text{Var}(t_N^-)}{\varepsilon^2 \cdot \mathbb{E}(t_N^-)} = \frac{\sum_{k=1}^N \frac{N(k-1)}{(N-K+1)^2}}{\varepsilon^2 \left(\sum_{k=1}^N \frac{N}{N-K+1} \right)^2}, \\ &= \frac{N}{\varepsilon^2 \cdot N^2} \cdot \frac{\sum_{k=1}^N \frac{(k-1)}{(N-k+1)^2}}{\left(\sum_{k=1}^N \frac{1}{N-k+1} \right)^2}. \end{aligned} \quad (2.1)$$

Sendo $N-1$ o maior número dos termos da soma $\sum_{k=1}^N (k-1)$, então a expressão em (2.1) é menor ou igual a

$$\frac{N(N-1)}{\varepsilon^2 \cdot N^2} \cdot \frac{\sum_{k=1}^{N-1} \frac{1}{k^2}}{\left(\sum_{k=1}^N \frac{1}{N-k+1} \right)^2} \leq \frac{N(N-1)}{\varepsilon^2 \cdot N^2} \cdot \frac{1 + \sum_{k=2}^{N-1} \frac{1}{k^2 - 1}}{(\log N)^2},$$

$$\leq \frac{N(N-1)}{\varepsilon^2 \cdot N^2} \cdot \frac{1 + \frac{3}{4} - \frac{2N+1}{2N(N+1)}}{(\log N)^2}.$$

Portanto,

$$\lim_{N \rightarrow \infty} \frac{N(N-1)}{\varepsilon^2 \cdot N^2} \cdot \frac{1 + \frac{3}{4} - \frac{2N+1}{2N(N+1)}}{(\log N)^2} = 0.$$

Portanto, $\lim_{N \rightarrow \infty} \mathbb{P} \left(\left| \frac{t_N^-}{\mathbb{E}(t_N^-)} - 1 \right| > \varepsilon \right) = 0.$ □

Corolário 2.1. *Sejam σ^ν e σ^ζ passeios homogêneos em H_N construídos simultaneamente, com configurações iniciais ν, ζ . Suponha para $0 \leq f \leq 1$, que $D_N(0) = \frac{[N \cdot f]}{N}$.*

Então, $\lim_{N \rightarrow \infty} \mathbb{P}(\sigma^\nu(t(N)) \neq \sigma^\zeta(t(N))) = 0$, para qualquer $t(N)$ com $\lim_{N \rightarrow \infty} \frac{t(N)}{N \log N} = \infty$.

Demonstração. Primeiros notamos que $t_N^- = \inf\{t > 0 : \{I(1), \dots, I(t)\} = \{1, \dots, N\}\}$. Assim,

$$\mathbb{P}(\sigma^\nu(t(N)) \neq \sigma^\zeta(t(N))) \leq \mathbb{P}(t_N^- > t(N)) \leq \frac{N(\log N + 1)}{t(N)}.$$

Portanto, por hipótese temos que

$$\lim_{N \rightarrow \infty} \mathbb{P}(\sigma^\nu(t(N)) \neq \sigma^\zeta(t(N))) = 0.$$

□

Corolário 2.2. *Seja $t(N)$ com $\lim_{N \rightarrow \infty} \frac{t(N)}{N \log N} = \infty$. Então,*

$$\lim_{N \rightarrow \infty} \left| \mathbb{P}(\sigma^+(t(N)) = \zeta) - \frac{1}{2^N} \right| = 0,$$

para todo $\zeta \in H_N$.

Demonstração. Vamos primeiro mostrar que μ é uma medida invariantes com respeito à evolução do passeio homogêneo.

$$\begin{aligned} \mathbb{P}(x_1 = \zeta) &= \sum_{\nu \in H_N} \mathbb{P}(x_1 = \zeta | x_0 = \nu) \cdot \frac{1}{2^N}, \\ &= \mathbb{P}(x_1 = \zeta | x_0 = \zeta) \cdot \frac{1}{2^N} + \sum_{i=1}^N \mathbb{P}(x_1 = \zeta | x_0 = \zeta^i) \cdot \frac{1}{2^N}, \\ &= \frac{1}{2} \cdot \frac{1}{2^N} + \frac{1}{2} \cdot \frac{1}{N} \cdot \frac{1}{2^N} \cdot N = \frac{1}{2^{N+1}} + \frac{1}{2^{N+1}} = \frac{1}{2^N} = \mathbb{P}(x_1 = \nu). \end{aligned}$$

Assim,

$$\begin{aligned}
\left| \mathbb{P}(\sigma^+(t(N)) = \zeta) - \frac{1}{2^N} \right| &= \left| \mathbb{P}(\sigma^+(t(N)) = \zeta) - \sum_{\nu \in H_N} \mu(\nu) \mathbb{P}((\sigma^\nu = \zeta)) \right|, \\
&\leq \sum_{\nu \in H_N} \mu(\nu) \left| \mathbb{P}(\sigma^+(t(N)) = \zeta) - \mathbb{P}(\sigma^\nu(t(N)) = \zeta) \right|, \\
&\leq \sum_{\nu \in H_N} \sup(\mathbb{P}(\sigma^+(t(N)) \neq \sigma^\nu(t(N)))) = 0.
\end{aligned}$$

A última igualdade é análoga ao corolário anterior. $\square$

Obs: Para um movimento não preguiçoso, $\xi(t)$, talvez não seja possível ocorrer um acoplamento. Se duas configurações iniciais, ν e ζ , diferem em um número ímpar de coordenadas e tem esse tipo de movimentação, nunca poderá haver um acoplamento, pois a distância mínima é, $D_N(t) \geq \frac{1}{N}$.

Demonstração. Seja $\nu, \phi \in H_N$ tal que $D_N^{\nu, \phi}(0) = n$ sendo n um número ímpar. Vamos observar 3 situações possíveis no passeio:

- 1) Se $(\nu^i) = (\phi^i)$, então $(n^i)_i = (\phi^i)_i$;
- 2) Se $(\nu^i) \neq (\phi^i)$, então $(n^i)_i \neq (\phi^i)_i$;
- 3) Se $(\nu^i) = (\phi^i)$, então $(n^i)_i \neq (\phi^i)_j$ sendo $i \neq j$.

Se ocorrer alguns desses casos, a distância das nossas cadeias, ou se mantém a mesma ou afastam-se, mas nunca se aproximam. Logo, supondo que por ventura só alteramos as entradas que são distintas ao longo do tempo. Como a cada passo, $I_\nu(1) = i$ e $I_\phi(1) = j$ tal que $i \neq j$, então $D_N^{\nu, \phi}(1) = n - 2$ já que estamos considerando sempre o melhor dos casos, que seria mudar duas entradas diferentes. Repetindo isso várias vezes, temos que existe um t' tal que $D_N^{\nu, \phi}(t') = 1$, pois n é ímpar e a cada tempo diminuimos duas posições. Assim, caímos no caso 3), ou seja, $D_N^{\nu, \phi}(t) \leq \frac{1}{N}$. $\square$

O Teorema 2.2 nos diz que é certo que $\xi(t)$ retorna a uma posição visitada quando $N \rightarrow \infty$, além de nos mostrar que essa posição de retorno é do tipo $\xi(i)$ e $\xi(j) \forall i, j \leq k + 1$, e $\xi(k + 2) = \xi(k)$ para algum k .

Teorema 2.2. *Seja $S_N = \inf(k > 0 : \xi(k) \in \{\xi(0), \xi(1), \dots, \xi(k - 1)\})$ e $\Gamma_1 = \inf(k > 0 : I(k) = I(k - 1))$. Então, $\lim_{N \rightarrow \infty} \mathbb{P}[S_N = \Gamma_1] = 1$.*

Para a demonstração desse teorema vamos adotar algumas definição e notações:

- 1) Seja I uma variável que escolhe uma coordenada i de um $\nu \in H_N$. Vamos denotar I avaliado no tempo k como I_k .
- 2) $J_1 = \{(I_1, I_2) \in \{1, 2, \dots, N\}^2 : I_1 = I_2\}$.
- 3) $J_l = \{(I_1, I_2, \dots, I_{2l}) \in \{1, 2, \dots, N\}^{2l} : \sum_{k=1}^{2l} 1_{\{i=I_k\}} \in \{0, 2, 4, \dots, 2l\} \forall i \in \{1, 2, \dots, N\} \text{ e } (I_1, I_2, \dots, I_{2k}) \notin J_k \forall k \in \{1, 2, \dots, l-1\}\}$.
- 4) Definimos um grupo de $i \in \{1, 2, \dots, N\}$ do vetor $(I_1, I_2, \dots, I_{2l}) \in J_l$ como $\{k \in \{1, 2, \dots, 2l-1, 2l\} : i = I(k)\}$.

Uma forma de pensar nos vetores desses J 's é ver eles como caminho na forma de laço.

Proposição 2.1. Para $l \geq 3$

$$\mathbb{P}(I_1, \dots, I_{2l}) \in J_l \leq \frac{8}{N^3}.$$

Demonstração. Sabemos que $\mathbb{P}(I_1, \dots, I_{2l}) \in J_l = \frac{|J_l|}{N^{2l}}$. Fazendo uma estimativa superior temos que $|J_l| \leq 2N^{2l-2}$, pois sendo $(I_1, \dots, I_{2l}) \in J_l$, então $(I_1, \dots, I_{2l-2}) \notin J_{l-1}$, evitando a recursividade, podemos supor que $\exists i' \in \{1, \dots, N\}$ tal que $\sum_{k=1}^{2l-2} 1_{\{i'=I_k\}}$ é ímpar, como o tamanho do vetor é par, então deve $\exists i'' \neq i'$ tal que $\sum_{k=1}^{2l-2} 1_{\{i''=I_k\}}$ é ímpar. Logo, como $(I_1, I_2, \dots, I_{2l-1}, I_{2l}) \in J_l$, então $\sum_{k=1}^{2l} 1_{\{i'=I_k\}}$ e $\sum_{k=1}^{2l} 1_{\{i''=I_k\}}$ são somas pares, ou seja, os dois últimos índices são determinados pelos i' e i'' . Como i' e i'' são diferentes, então podemos permutá-los, assim $|J_l| \leq 2N^{2l-2}$. Podemos dividir J_l em dois conjuntos disjuntos, J_l'' e J_l' , onde J_l'' é o conjunto onde nas três últimas entradas existe duas iguais, e J_l' é o conjunto onde as três últimas entradas são distintas. Vamos achar uma estimativa superior para esse conjuntos. Em J_l' como a entrada $2l-2$ é distinta das duas últimas e sabemos que temos dois blocos ímpares distintos que determinam as duas últimas entradas, então o bloco de I_{2l-2} no subvetor $(I_1, I_2, \dots, I_{2l-3})$ é ímpar, pois se fosse o contrário, a soma $\sum_{k=1}^{2l} 1_{\{I_{2l-2}=I_k\}}$ seria ímpar. Logo, a entrada $2l-2$ é determinado por um terceiro bloco ímpar. Logo, $|J_l'| \leq 3!N^{2l-3}$. No conjunto J_l'' perceba que como $I_{2l-1} \neq I_{2l}$, então existem somente dois casos $I_{2l-2} = I_{2l}$ e $I_{2l-1} = I_{2l-2}$. Sem perda de generalidade, vamos supor que é verdade $I_{2l-1} \neq I_{2l}$. Como ambas as duas coordenadas estão determinadas pelos mesmo bloco, então podemos lidar com esse vetor de $2l$ entradas como se fosse de

$2l - 2$ entradas. Assim, $|J_l''| \leq N \cdot |J_{l-1}''|$, pois as coordenada repetida pode ter combinação de N valores. Portanto,

$$\frac{|J_l|}{N^{2l}} = \frac{|J_l'|}{N^{2l}} + \frac{|J_l''|}{N^{2l}} \leq \frac{3!N^{2l-3}}{N^{2l}} + \frac{N \cdot 2N^{2l-4}}{N^{2l}} = \frac{8}{N^3}.$$

□

Capítulo 3

Abordagem Potencial para Metaestabilidade

No capítulo anterior, estudamos o processo de metaestabilidade no contexto do hipercubo, onde provamos teoremas base da natureza desse processo. Neste capítulo, iremos estudar o processo de metaestabilidade abordando a ideia de potências.

3.0.1 Caminhar aleatório em malhas

Inicialmente temos um exemplo de um processo de Cadeia de Markov em malhas conectadas que nos elucida a motivação da definição de metaestabilidade. Denotamos E_N , $N \geq 1$, o conjunto de estados de uma malha quadrada de largura N e dimensão d , como representado abaixo. Os estados (pontos ou configurações) de E_N é representado por η, ξ, ζ .

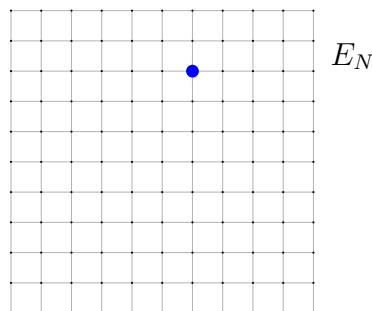


Figura 3.1: Malha $\Lambda_N = \{0, \dots, N\}^d$ com um elemento de E_N destacado por azul.

Se considerarmos uma evolução dos estados podemos indexar E_N para cada tempo, $E_{t,N}$. Para isso, consideremos que para $E_{t,N}$, temos um sequência de malhas Λ_N onde para cada par de malhas vizinhas temos somente um ponto em comum, como a figura à abaixo.

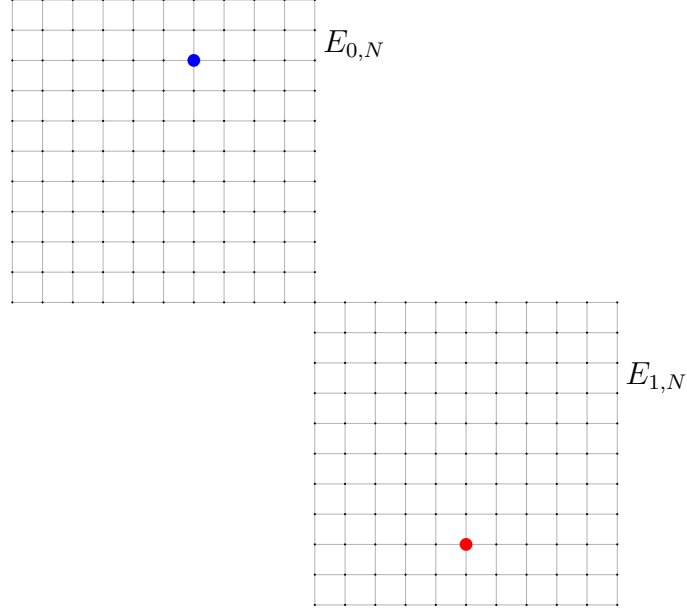


Figura 3.2: Um caminhar aleatório com dois estados

Assim podemos definir E_N como a união dos $E_{t,N}$, ou seja, podemos pensar E_N como os estados de uma partícula de $0 \leq t \leq p$ sendo p um inteiro. Vamos denotar $\eta_N(t)$ uma cadeia de Markov de tempo contínuo em E_N tal que o pulo de $\eta \in E_{t,N}$ para seus vértices vizinhos é uniforme, e para mudar seu estado espera um tempo exponencial. Perceba que essa cadeia de Markov é irreduzível.

Dizemos que o grau de $\eta \in E_N$, $\deg(\eta)$, é quantidade de vizinhos. Assim conseguimos definir uma medida de probabilidade

$$\pi_N(\eta) = \frac{\deg(\eta)}{Z_N}$$

onde $Z_N = \sum_{\xi \in E_N} \deg(\xi)$. Então, como sabemos que π_N satisfaz a *Equação de balanço*, a distribuição é estacionária e única. Como síntese da evolução da cadeia de Markov, utilizaremos o *Coarse-grained-model* que permite simplificar a representação do estado da partícula. Vamos denotar $\Gamma_N : E_N \rightarrow \{0, 1, 2, 3\}$ a projeção de uma partícula no seu tempo e χ a função característica,

$$\Gamma_N(\eta) = \sum_{k=0}^3 k \chi_{E_{k,N}}(\eta).$$

Perceba que não tem problema nas intersecções dos quadrados, pois cada quadrado irá ter $(N^d - 1)$ vértices. A função da evolução assintótica do *Coarse-grained-model*

$$Y_N(t) = \Gamma_N(\eta_N(t))$$

é um passeio aleatório entre cubos discretos. Vamos denotar $z_N(t)$ o andar aleatório simétrico, de tempo contínuo em Λ^N , ou seja, o processo η_N restrito a Λ_N , ademais seja $\pi_N(\Lambda_N)$ a medida estacionária em Λ_N . Pelo Teorema 1.3, temos que a ordem com que $z_N(t)$ converge para uma transição estacionária é de N^2 , além disso o tempo estimado de um ponto x para outro que dista N é da ordem $\alpha_N = N^2 \log N$ em dimensão $d = 2$ e $\alpha_N = N^d$ em dimensão $d \geq 3$.

Assumindo que iniciamos no centro de $\mathbb{Z}_N^d$, definimos B como o conjunto dos pontos de intersecção $E_{j,N} \cap E_{i,N}$ para todo $j \neq i$. Ademais, tomemos

$$H_B^N = \inf\{t \geq 0 : \eta_N(t) \in B\}.$$

Como a ordem de H_B^N é maior que a ordem de convergência da cadeia para sua estacionária, então dizemos que ela termaliza ou equilibra antes de chegar nas quinas compartilhadas. Assim podemos intuir que quanto mais perto de B mais nossa partícula esquece do caminho que veio, consequentemente a probabilidade da partícula mudar de estado em B se a próxima de $\frac{1}{2}$. Definindo uma sequência especial podemos descrever o que ocorre com a partícula ao se aproximar de ξ . Seja $(\ell_N : N \geq 1)$ de números reais tais que $\ell_N \rightarrow \infty$ e $\frac{\ell_N}{N} \rightarrow 0$, criamos V_N , o conjunto dos pontos que distam até ℓ_N de ξ . Tomando o momento que a partícula chega a fronteira de V_N , sabemos que a ordem de tempo de sair de V_N é $\ell_N^2 \log(\ell_N^2)$, uma medida muito menor que a ordem da partícula alcançar o ξ . Assim ao adentrar em V_N a partícula realiza uma caminhada de tempo irrisório, chegando e retornando em ξ , passando em $E_{j,N}$ ou $E_{j\pm 1,N}$. Como o tempo de convergência da distribuição em V_N é ℓ_N^2 , ao entrar no quadrado, em pouco tempo o processo é termalizado. Pensando nesse processo, como o tempo dentro de V_N e o *mixing time* são insignificantes em relação a escala do tempo de escape, podemos adequadamente ignorar essa passagem por V_N e o tempo até a convergência da distribuição, permitindo que a cadeia de Markov cresça na mesma ordem do tempo de escape α_N :

$$\mathbf{Y}_N(t) = Y_N(t\alpha_N) = \Gamma_N(\eta_N(t\alpha_N))$$

o qual tem imagem em $S = \{0, 1, 2, 3\}$, composta pela cadeia $Y_N(t)$ de tempo contínuo de *holding time* escritamente positivo e com $p(j, j \pm 1) = \frac{1}{2}$

3.1 Simulações em python

Como base para entendermos o escopo deste trabalho, foi criado algumas simulações usando a linguagem Python, fazendo uso de bibliotecas como *matplotlib*, *Pandas* e *Numpy*. Inicialmente falamos do caminhar aleatório com acoplamento

estudado pelos Adilson Simonis e Cláudia Peixoto em ??, em seguida foi criado um modelo de metaestabilidade com quatro estados diferentes, descrito por Claudio Ladim. Por último temos a demonstração para onde converge a distribuição da variável que observa o tempo de saída da partícula de um estado.

3.1.1 Caminhar aleatório com acoplamento

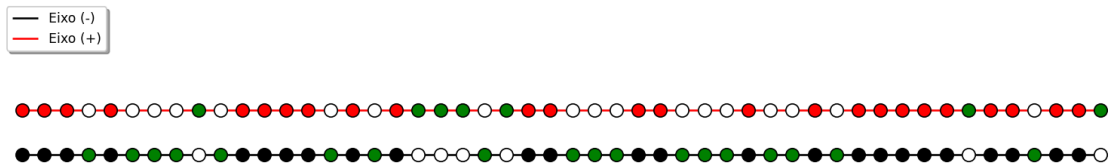


Figura 3.3: Acoplamento de dois passeios aleatórios

O código apresentado cria uma simulação com visualização gráfica cujo objetivo principal é ilustrar o comportamento de acoplamento (coupling) entre duas configurações aleatórias ao longo do tempo. Para isso, são utilizadas bibliotecas como NumPy para manipulação eficiente de vetores, Matplotlib para a construção da interface gráfica e animação.

Inicialmente, é definido um tamanho fixo $TAM=50$, que representa o número de posições do sistema. Cada posição pode assumir um valor de "spin", isto é, $+1$ ou -1 , o que remete ao modelo de Ising. Duas configurações independentes são criadas a partir da classe `Configuracao`, uma iniciada com todos os spins positivos e outra com todos negativos. Essas configurações são armazenadas como listas, e sua representação visual é feita por meio de pontos (as "bolinhas") no gráfico.

Um ponto central do código é o uso de atributos de classe, como *icouple* e *disponiveis*. O conjunto *icouple* armazena os índices das posições onde as duas configurações coincidem, ou seja, onde já ocorreu o acoplamento. Esse mecanismo é essencial para o entendimento do código, pois, uma vez que duas posições forem acopladas, suas evoluções futuras passam a ser sincronizadas.

A dinâmica do sistema é implementada no método `andar`, que representa um passo temporal da simulação. A cada iteração, dois índices aleatórios são selecionados. Para cada configuração, o valor do spin nesses índices é atualizado de forma aleatória. Porém como dito anteriormente, se o índice já pertence ao conjunto *icouple*, a atualização é feita de forma conjunta nas duas configurações, garantindo que permaneçam iguais naquela posição. Caso contrário, as atualizações ocorrem de maneira independente. Após essa etapa, o algoritmo verifica novamente todas

as posições para identificar novos acoplamentos.

O método visualizar desempenha um papel fundamental na compreensão do processo. O método visualizar utiliza cores para indicar o estado de cada posição: vermelho e preto representam spins positivos e negativos, respectivamente, enquanto verde indica posições onde ocorreu acoplamento. Assim, ao longo da animação, é possível observar a propagação gradual dessas regiões verdes, evidenciando a convergência entre as duas configurações.

A animação é controlada pela função `update`, utilizada em conjunto com a classe `FuncAnimation` do Matplotlib. A cada quadro, o tempo é incrementado e um novo passo da dinâmica é executado. O processo continua até que todas as posições estejam acopladas, isto é, quando o tamanho do conjunto `icouple` atinge TAM . Nesse momento, a simulação é interrompida e são exibidas informações relevantes, como o tempo total necessário para o acoplamento completo e uma estimativa teórica baseada em $TAM \cdot \log(TAM)$, que é teorizada pelo Teorema 2.1.

```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import numpy as np
import random as rd
import math

TAM = 50  # constante do tamanho do vetor
Tempo: int = 0  # tempo

# Definindo o intervalo do eixo x
x = np.arange(0, TAM, 1)

class Configuracao:
    icouple = set()  # conj. de indice que estão em couple
    disponiveis = [i for i in range(TAM)]  # conj. de indice

    def __init__(self, spin: int, pos, neg):
        # construtor criando um configuração e inicializando
        # com o spin (+ ou -)
        self.conf = [spin for i in range(TAM)]
        if spin > 0:
            for i in range(0, TAM):
                pos[i].set_facecolor("red")
        else:
            for i in range(0, TAM):
                neg[i].set_facecolor("black")
```



```

@classmethod
def visuaizar(cls, p1, p2, bolinhas1, bolinhas2):
    for i in range(0, TAM):
        if p1.conf[i] == p2.conf[i]: # procurando o couple
            if p1.conf[i] > 0:
                bolinhas1[i].set_facecolor("green")
                bolinhas2[i].set_facecolor("white")
            else:
                bolinhas1[i].set_facecolor("white")
                bolinhas2[i].set_facecolor("green")
        else:
            if p1.conf[i] > 0:
                bolinhas1[i].set_facecolor("red")
            else:
                bolinhas1[i].set_facecolor("black")
            if p2.conf[i] > 0:
                bolinhas2[i].set_facecolor("red")
            else:
                bolinhas2[i].set_facecolor("black")

@classmethod
def andar(cls, p1, p2, bolinhas1, bolinhas2):
    # Move uma posição aleatória
    i1 = rd.choice(Configuracao.disponiveis)
    # escolhendo o indice
    i2 = rd.choice(Configuracao.disponiveis)

    if i1 in Configuracao.icouple:
        p1.conf[i1] = rd.choice([1, -1])
        # escolhendo o spin do indice in couple
        p2.conf[i1] = p1.conf[i1]
    elif i1 not in Configuracao.icouple:
        p1.conf[i1] = rd.choice([1, -1])
        # escolhendo o spin do indice

    if i2 in Configuracao.icouple:
        p2.conf[i2] = rd.choice([1, -1])
        # escolhendo o spin do indice in couple
        p1.conf[i2] = p2.conf[i2]
    elif i2 not in Configuracao.icouple:

```

```

p2.conf[i2] = rd.choice([1, -1])
# escolhendo o spin do indice

for i in range(0, TAM):
    # adiciona os indices que fizeram couple
    if p1.conf[i] == p2.conf[i]:
        Configuracao.icouple.add(i)

Configuracao.visuaizar(p1, p2, bolinhas1, bolinhas2)

# === Definição do matplotlib ===
fig, ax = plt.subplots(figsize=(8, 4))

# Caminho 1
ax.hlines(y=1, xmin=0, xmax=TAM - 1,
color="black", zorder=1, label="Eixo (-)")
ax.hlines(y=3, xmin=0, xmax=TAM - 1,
color="red", zorder=1, label="Eixo (+)")

bolinhas2 = [ax.scatter(xi, 1, facecolors="white",
edgecolors="black", s=100, zorder=2) for xi in x]
bolinhas1 = [ax.scatter(xi, 3, facecolors="white",
edgecolors="black", s=100, zorder=2) for xi in x]

# Ajustes do gráfico
ax.set_xlim(-1, TAM)
ax.set_ylim(0, 8)
ax.set_aspect("equal", adjustable="box")
ax.axis("off")
ax.legend(loc='upper left', frameon=True, fancybox=True, shadow=True)

# === Função de atualização da animação ===
p1 = Configuracao(+1, bolinhas1, bolinhas2)
p2 = Configuracao(-1, bolinhas1, bolinhas2)

def update(frame):
    global Tempo
    Tempo += 1
    Configuracao.andar(p1, p2, bolinhas1, bolinhas2)
    if len(Configuracao.icouple) == TAM:
        print("\n" + "="*50)

```

```

print("TODAS AS BOLINHAS ESTÃO VERDES!")
print("="*50)
print(f"Tempo total: {Tempo} segundos")
print(f"Tempo esperado: {format(math.log(TAM)*TAM, '.2f')} segundos")
print("="*50)
anim.event_source.stop()
return bolinhas1 + bolinhas2

# === Criação da animação ===
anim = FuncAnimation(fig, update, frames=200,
interval=100, blit=False, repeat=True)

plt.show()

```

3.1.2 Metaestabilidade: Uma abordagem com potencial

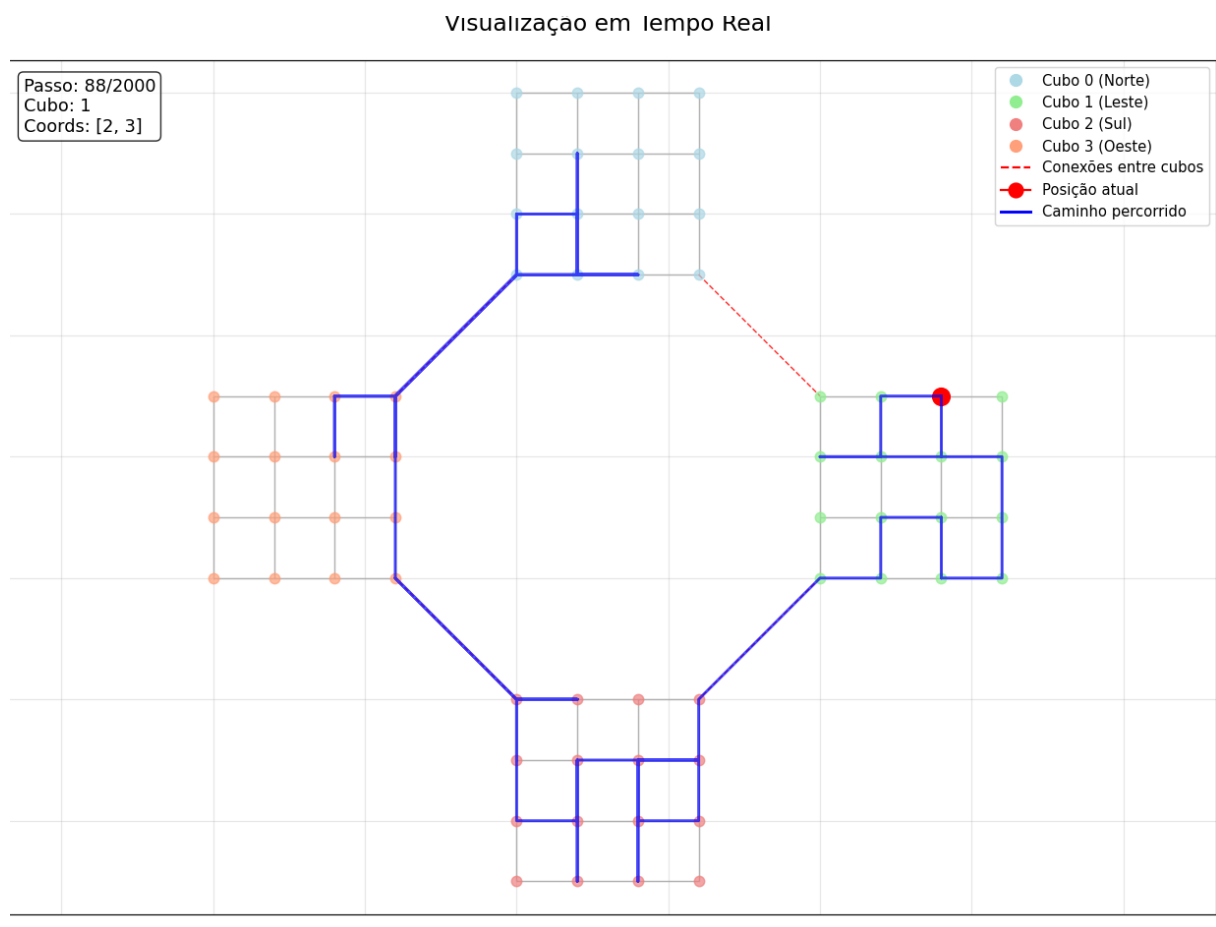


Figura 3.4: Andar mestaestável com 4 estados

O código apresentado implementa uma simulação de passeio aleatório em uma estrutura de grafo especialmente construída para evidenciar o fenômeno de metaestabilidade. A modelagem combina conceitos de probabilidade, grafos e visualização computacional, utilizando bibliotecas como *NumPy*, *Matplotlib* e *NetworkX*. A ideia central é representar o espaço de estados como uma rede de regiões conectadas (os "cubos"), mas com poucas conexões entre si, o que induz um comportamento típico de sistemas metaestáveis: longos períodos de permanência em uma região antes de transições raras para outra.

A classe *MetastableRandomWalk* é responsável por construir e gerenciar essa estrutura. No método `'_grafo'`, o espaço de estados é definido como um grafo composto por quatro "cubos"(d)-dimensionais, cada um contendo (N^d) vértices. Cada vértice representa um estado possível do sistema e é identificado por um rótulo que indica tanto o cubo ao qual pertence quanto sua posição interna. A conversão entre índices lineares e coordenadas multidimensionais é tratada pelos métodos `'_indexCoord'` e `'_coordIndex'`, permitindo organizar os vértices como se estivessem em uma grade discreta.

Dentro de cada cubo, os vértices são conectados aos seus vizinhos imediatos, formando uma malha regular. Essa densa conectividade interna garante que o passeio aleatório se mova facilmente dentro de um mesmo cubo. Por outro lado, o método `'_JuntarCubo'` cria apenas algumas conexões específicas entre cubos distintos, ligando pontos de canto. Essa escolha é crucial: ela cria "gargalos" estruturais no grafo, dificultando a transição entre regiões diferentes e, consequentemente, gerando o comportamento metaestável.

A dinâmica do passeio aleatório é definida de maneira simples e clássica. O método `'inicio'` escolhe um estado inicial dentro de um cubo específico, com uma restrição adicional, o ponto deve estar em uma região mais central do cubo, evitando bordas. Essa escolha reforça o efeito metaestável, pois aumenta a probabilidade de o sistema permanecer inicialmente dentro do mesmo cubo. Já o método `'andar'` implementa o passo do passeio aleatório, no qual o próximo estado é escolhido uniformemente entre os vizinhos do estado atual.

A trajetória do sistema é armazenada ao longo do tempo, permitindo acompanhar o caminho percorrido. Esse registro é essencial para a visualização e para a análise do comportamento do processo. O método `'simular'` apenas automatiza a execução de múltiplos passos consecutivos, retornando a sequência de estados visitados.

A visualização é tratada pela classe `'visualizar'`, que constrói uma representação bidimensional do grafo. O método `'_nodeCoord'` projeta os diferentes cubos em posições distintas do plano, organizando-os como regiões separadas (norte, sul, leste e oeste). Isso facilita a interpretação visual das transições entre cubos.

A função `'setup_plot'` desenha os nós e arestas, diferenciando as conexões internas (em cinza) das conexões entre cubos (em vermelho tracejado), além de configurar elementos como legenda e título.

A animação é controlada por meio da função `'animate_step'`, que atualiza, a cada quadro, tanto a posição atual do passeio quanto o caminho acumulado. O ponto vermelho indica o estado atual, enquanto a linha azul mostra a trajetória já percorrida. Informações adicionais, como o número do passo, o cubo atual e as coordenadas internas, são exibidas dinamicamente. A utilização de `'FuncAnimation'` permite observar em tempo real a evolução do sistema.

Do ponto de vista conceitual, o código ilustra de forma clara o fenômeno de metaestabilidade: o passeio aleatório tende a permanecer por longos períodos dentro de um mesmo cubo (devido à alta conectividade interna), realizando apenas ocasionalmente transições para outros cubos (devido às poucas conexões entre eles). Esse comportamento é característico de sistemas com múltiplos poços de energia ou regiões de estabilidade local, sendo amplamente estudado em áreas como física estatística, teoria das cadeias de Markov e algoritmos de amostragem.

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import networkx as nx
import math
from matplotlib.animation import FuncAnimation

class MetastableRandomWalk:
    def __init__(self, N, d=2):
        self.N = N
        self.d = d
        self.EN = self._grafo()
        self.estadoAtual = None
        self.passo = 0
        self.caminho = []

    def _grafo(self):
        G = nx.Graph()
        # Cria os 4 cubos (E0, E1, E2, E3)
        cuboPonto = {}

        # Para cada cubo (0: norte, 1: leste, 2: sul, 3: oeste)
        for cubo in range(4):
            cuboPonto[cubo] = []
```

```

        # Gera pontos do cubo d-dimensional da vez cube
        for i in range(self.N**self.d):
            coords = self._indexCoord(i)
            NodeId = f"{cubo}_{i}"
            G.add_node(NodeId, cubo=cubo, coords=coords)
            cuboPonto[cubo].append(NodeId)

    # Conecta as vizinhas de cada cubo dentro de cada curbo,
    #mas não os pontos de intersecção
    for cubo in range(4):
        points = cuboPonto[cubo]
        for i, nodeOrin in enumerate(points):
            coords1 = self._indexCoord(i)
            # Encontra vizinhos no cubo
            for dim in range(self.d):
                for delta in [-1, 1]:
                    vizin = coords1.copy()
                    vizin[dim] += delta
                    if 0 <= vizin[dim] < self.N:
                        j = self._coordIndex(vizin)
                        node2 = points[j]
                        G.add_edge(nodeOrin, node2)

    # Conecta cubos adjacentes (pontos de canto compartilhados)
    self._JuntarCubo(G, cuboPonto)

    return G

def _indexCoord(self, index):
    #Converte índice linear para coordenadas d-dimensionais
    coords = []
    vindex = index
    for _ in range(self.d):
        coords.append(vindex % self.N)
        vindex //= self.N
    return coords

def _coordIndex(self, coords):
    #Converte coordenadas d-dimensionais para índice linear
    index = 0

```

```

potDim = 1
for i, coord in enumerate(coords):
    index += coord * potDim
    potDim *= self.N
return index

def _JuntarCubo(self, G, vCuboPontos):
    #Ajunção dos cubos
    conecta = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # pares de cubos adjacentes

    conectaPonto = [(self.N-1, 0),
                    (0, self.N-1), (0, 0), (self.N-1, self.N-1),
                    (0, self.N-1), (self.N-1, 0), (self.N-1, self.N-1),
                    (0, 0)] #Quem queremos ligar

    for i in range(0, 8, 2):
        pontos1 = vCuboPontos[conecta[int(i/2)][0]]
        pontos2 = vCuboPontos[conecta[int(i/2)][1]]

        indexNode1 = self._coordIndex(conectaPonto[i])
        indexNode2 = self._coordIndex(conectaPonto[i+1])

        print(indexNode1, indexNode2)

        G.add_edge(pontos1[indexNode1], pontos2[indexNode2])

def inicio(self, start=0):
    #Inicializa o passeio aleatório em um cubo
    cuboPontos= [node for node in self.EN.nodes
    if self.EN.nodes[node]['cubo'] == start]
    procurando = True
    while(procurando):
        procurando = False
        self.estadoAtual = np.random.choice(cuboPontos)
        for i in range(2):
            if(self.EN.nodes[self.estadoAtual]['coords'][i] > self.N -
            ( math.floor( math.sqrt(self.N) ) )/2 ):
                #para começar no quadrado interno
                procurando = True
                break

```

```

        elif (self.EN.nodes[self.estadoAtual]['coords'][i] <
              ( math.floor( math.sqrt(self.N) ) )/2 ):
            procurando = True
            break
    self.passo = 0
    self.caminho = [self.estadoAtual]

def andar(self):
    vizin = list(self.EN.neighbors(self.estadoAtual))
    self.estadoAtual = np.random.choice(vizin)
    self.passo += 1
    self.caminho.append(self.estadoAtual)

    return self.estadoAtual

def simular(self, steps, start=0):
    self.inicio(start)

    for _ in range(steps):
        self.andar()

    return self.caminho

def _nodeCoord(self):
    #Calcula posições dos nós para visualização 2D
    pos = {}
    cuboMeio = {
        0: (0, self.N + 1),      # Norte
        1: (self.N + 1, 0),      # Leste
        2: (0, -self.N - 1),     # Sul
        3: (-self.N - 1, 0)      # Oeste
    }

    for node in self.EN.nodes:
        cubo = self.EN.nodes[node]['cubo']
        coords = self.EN.nodes[node]['coords']
        projX, projY = cuboMeio[cubo]
        pos[node] = (coords[0] + projX, coords[1] + projY)

    return pos

```



```

class visualizar:
    def __init__(self, random_walk, iter=200):
        self.rw = random_walk
        self.iter = iter
        self.fig, self.ax = plt.subplots(figsize=(12, 10))
        self.pos = self.rw._nodeCoord()

        self.cubo_colors = ['lightblue', 'lightgreen',
                             'lightcoral', 'lightsalmon'] # Cores para diferentes cubos
        self.atualPonto = None
        self.line = None
        self.textPasso = None

    def setup_plot(self):
        """Configura o plot inicial"""
        self.ax.clear()

        # Desenha os cubos
        for cubo in range(4):
            # Nos do cubo
            nodeCubos = [node for node in self.rw.EN.nodes
                          if self.rw.EN.nodes[node]['cubo'] == cubo]

            # Desenha arestas do cubo
            arestasCubo = [(u, v) for u, v in self.rw.EN.edges()
                            if u in nodeCubos and v in nodeCubos]

            nx.draw_networkx_edges(self.rw.EN, self.pos,
                                   edgelist=arestasCubo,
                                   ax=self.ax, edge_color='gray', alpha=0.6)

            # Desenha nós do cubo
            nx.draw_networkx_nodes(self.rw.EN, self.pos,
                                   nodelist=nodeCubos, node_color=self.cubo_colors[cubo],
                                   node_size=50, alpha=0.7, ax=self.ax)

        # Desenha arestas entre cubos (conexões)
        arestasCuboInter = []
        for u, v in self.rw.EN.edges():
            cubo_u = self.rw.EN.nodes[u]['cubo']
            cubo_v = self.rw.EN.nodes[v]['cubo']

```

```

        if cubo_u != cubo_v:
            arestasCuboInter.append((u, v))

nx.draw_networkx_edges(self.rw.EN, self.pos,
                        edgelist=arestasCuboInter,
                        ax=self.ax, edge_color='red', style='--',
                        alpha=0.8)

# Configurações do plot
self.ax.set_title('Passeio Aleatório Metaestável \n Visualização
em Tempo Real', fontsize=16, pad=20)
self.ax.axis('equal')
self.ax.grid(True, alpha=0.3)

# Legenda
legend_elements = [
    plt.Line2D([0], [0], marker='o', color='w',
               markerfacecolor='lightblue',
               markersize=10, label='Cubo 0 (Norte)'),
    plt.Line2D([0], [0], marker='o', color='w',
               markerfacecolor='lightgreen',
               markersize=10, label='Cubo 1 (Leste)'),
    plt.Line2D([0], [0], marker='o', color='w',
               markerfacecolor='lightcoral',
               markersize=10, label='Cubo 2 (Sul)'),
    plt.Line2D([0], [0], marker='o', color='w',
               markerfacecolor='lightsalmon',
               markersize=10, label='Cubo 3 (Oeste)'),
    plt.Line2D([0], [0], color='red', linestyle='--',
               label='Conexões entre cubos'),
    plt.Line2D([0], [0], marker='o', color='red', markersize=10,
               label='Posição atual'),
    plt.Line2D([0], [0], color='blue', linewidth=2, label='Caminho
percorrido')
]

self.ax.legend(handles=legend_elements, loc='upper right')

# Inicializa elementos de animação
self.atualPonto, = self.ax.plot([], [], 'ro', markersize=12,
                                label='Posição Atual')
self.line, = self.ax.plot([], [], 'b-', alpha=0.7, linewidth=2,

```

```

label='Caminho')
self.textPasso = self.ax.text(0.02, 0.98, '',
transform=self.ax.transAxes,
                                fontsize=12, verticalalignment='top',
                                bbox=dict(boxstyle='round',
                                facecolor='white', alpha=0.8))

return self.atualPonto, self.line, self.textPasso

def animate_step(self, frame):
    """Atualiza a animação para cada frame"""
    if frame == 0:
        # Reinicia a simulação
        self.rw.inicio(0)
        self.rw.caminho = [self.rw.estadoAtual]
    elif frame <= self.iter:
        # Executa um passo
        self.rw.andar()

    # Atualiza visualização
    if self.rw.caminho:
        # Pega coordenadas do caminho
        camCoord = [self.pos[node] for node in self.rw.caminho]
        x_vals, y_vals = zip(*camCoord) if camCoord else ([], [])
        #faz a projeção dos x em x_vals e de y em y_vals como lista

        # Atualiza linha do caminho
        self.line.set_data(x_vals, y_vals)

        # Atualiza ponto atual
        posAtual = self.pos[self.rw.estadoAtual]
        self.atualPonto.set_data([posAtual[0]], [posAtual[1]])

        # Atualiza texto
        cubo_atual = self.rw.EN.nodes[self.rw.estadoAtual]['cubo']
        coords_atual = self.rw.EN.nodes[self.rw.estadoAtual]['coords']
        self.textPasso.set_text(f'Passo: {frame}/{self.iter}\nCubo:
        {cubo_atual}\nCoords: {coords_atual}')

    return self.atualPonto, self.line, self.textPasso

```

```

def animate(self, interval=100):
    """Cria e exibe a animação"""
    self.setup_plot()
    anim = FuncAnimation(self.fig, self.animate_step,
        frames=self.iter+1,
                        interval=interval, blit=True, repeat=True)
    plt.tight_layout()
    plt.show()
    return anim

if __name__ == "__main__":
    # Parâmetros da simulação
    N = 4 # Tamanho do cubo
    d = 2 # Dimensão
    iter = 2000 # Número de passos para animação

    print("Criando passeio aleatório metaestável...")
    rw = MetastableRandomWalk(N, d)

    print("Iniciando visualização animada...")
    print("Configuração:")
    print(f"- {N}x{N} pontos por cubo")
    print(f"- 4 cubos conectados")
    print(f"- {iter} passos da animação")
    print(f"\nA animação será iniciada...")

    visualizer = visualizar(rw, iter)
    anim = visualizer.animate(interval=150) # Intervalo em milissegundos

```

3.1.3 Convergência de distribuição: Tempo de saída

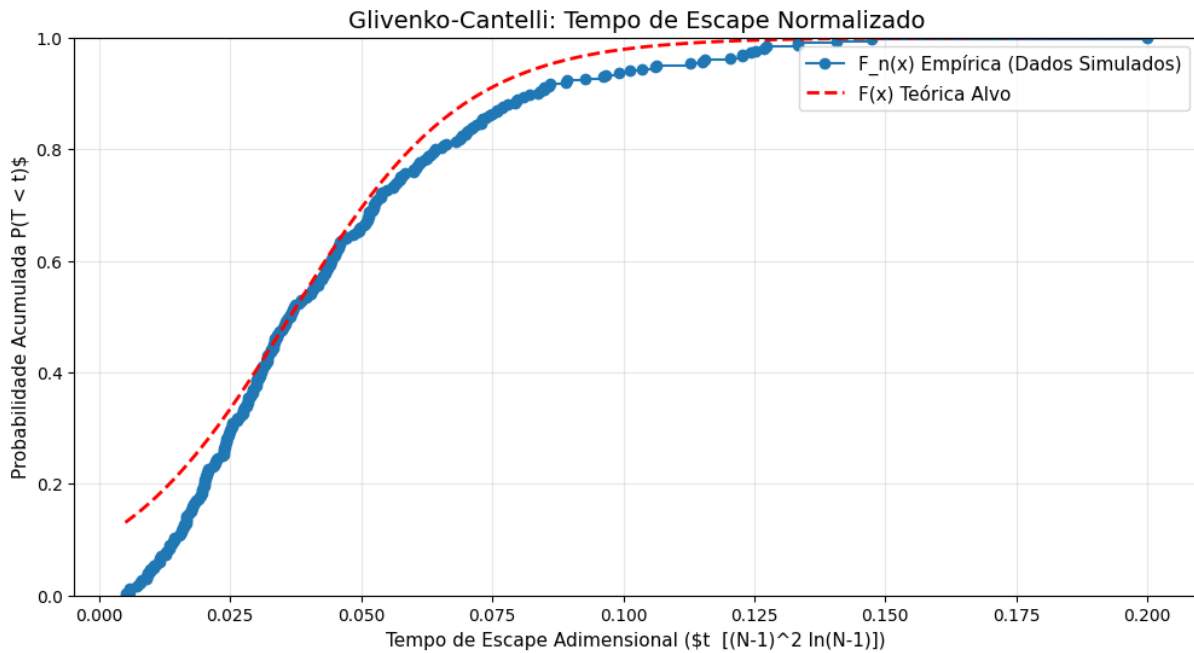


Figura 3.5: Convergência da distribuição do tempo de fuga de um estado

Nesta simulação é apresentado a modelagem de um caminhar aleatório em uma malha bidimensional quadrada de tamanho $N \times N$, estruturada na forma de um grafo. A simulação afere o tempo de saída do estado (a malha) que a partícula se encontra, e demonstra para qual convergência a distribuição da observação desse evento tende. Diferente do modelo anterior que foi construído 4 malhas conectadas utilizando conceitos básicos de grafos, como nesse modelo temos somente uma malha, podemos utilizar o método `'grid_2d_graph'` da biblioteca `'NetworkX'` para criar tal objeto. A descrição do modelo é a mesma do anterior, começando no meio do estado, transição uniforme, e com o tempo de termalização menor do que o tempo de chegar na borda para a transição de estado. Um ponto importante a se salientar sobre o código, é o método para indicar que a partícula saiu do estado é uma borda ao redor do quadrado $N \times N$, ou seja, o método cria um grid para $N+1$, mas o estado é de N , o resto é uma adaptação para representar a saída do estado. Por fim, para representar e verificar a comparação da convergência da distribuição, utilizamos o teorema de Glivenko-Cantelli para intuir no resultado da convergência, e na simulação utilizamos a biblioteca `'scipy.stats'`, focada em probabilidade e estatística, para plotar o gráfico da Logística e realizar a comparação.

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
```

```
import math
```

```
class SingleCubeEscape:
```

```
    def __init__(self, N):
```

```
        self.N = N
```

```
        self.G = self._build_graph()
```

```
        self.reset()
```

```
        self.escapes = []    # tempos de saída
```

```
        self.attempt = 0
```

```
    def _build_graph(self):
```

```
        # Cria um grid NxN
```

```
        return nx.grid_2d_graph(self.N, self.N)
```

```
    def reset(self):
```

```
        self.passo = 0
```

```
        self.caminho = []
```

```
        mid = self.N // 2 # Começa sempre no centro
```

```
        self.estadoAtual = (mid, mid)
```

```
        self.caminho.append(self.estadoAtual)
```

```
    def is_boundary(self, node):
```

```
        x, y = node
```

```
        return x == 0 or y == 0 or x == self.N - 1 or y == self.N - 1
```

```
    def andar(self):
```

```
        vizinhos = list(self.G.neighbors(self.estadoAtual))
```

```
        self.estadoAtual = vizinhos[np.random.randint(len(vizinhos))]
```

```
        self.passo += 1
```

```
        self.caminho.append(self.estadoAtual)
```

```
        # Condição de escape
```

```
        if self.is_boundary(self.estadoAtual):
```

```
            # Normalização de escala para malha 2D
```

```
            fator_escala = ((self.N - 1) ** 2 * math.log(self.N - 1))
```

```
            self.escapes.append(self.passo / fator_escala)
```

```
            self.attempt += 1
```

```
            self.reset()
```

```

def simular(self, tentativas):
    for i in range(tentativas):
        while True:
            self.andar()
            if self.attempt > i:
                break
        print(f"Tentativa {i+1}/{tentativas} concluída.")
    return self escapes

def plot_resultados(escapes):
    plt.figure(figsize=(12, 6))

    escapes_ordenados = np.sort(escapes)

    plt.ecdf(escapes_ordenados, marker='o', label='F_n(x)
    Empírica (Dados Simulados)', color='tab:blue')

    x_teorico = np.linspace(escapes_ordenados[0], escapes_ordenados[-1], 200)

    from scipy.stats import logistic # específico para distribuição
    loc_estimado = np.median(escapes_ordenados)
    scale_estimado = np.std(escapes_ordenados) * 0.55
    cdf_teorica = logistic.cdf(x_teorico, loc=loc_estimado,
    scale=scale_estimado)

    plt.plot(x_teorico, cdf_teorica, color='red', linestyle='--', linewidth=2
    label='F(x) Teórica Alvo')

    plt.title("Glivenko-Cantelli: Tempo de Escape Normalizado", fontsize=14)
    plt.xlabel("Tempo de Escape Adimensional
    ( $t \sqrt{\ln(N-1)/(N-1)}$ )", fontsize=11)
    plt.ylabel("Probabilidade Acumulada  $P(T < t)$ ", fontsize=11)
    plt.grid(True, alpha=0.3)
    plt.legend(fontsiz
    e=11)
    plt.show()

if __name__ == "__main__":
    N = 1000
    tentativas = 200

```

```
sim = SingleCubeEscape(N + 1)

print("Rodando simulação...")
escapes = sim.simular(tentativas)

plot_resultados(escapes)
```


Referências Bibliográficas

- [Kipnis-Landim] C. Kipnis and C. Landim, *Scaling Limits of Interacting Particle Systems* , Springer-Verlag Berlin Heidelberg, (1999).
- [Tsallis] Tsallis, Constantino , *Introduction to Nonextensive Statistical Mechanics*, New York, (2009).
- [Shannon] Shannon, C. E. , *A Mathematical Theory of Communication* , The Bell System Technical Journal, (1948).
- [Khinchin] Khinchin, A. I. , *Mathematical Foundations of Information Theory* , New York, (1957).
- [Durrett] Durrett, Richard. , *Probability, Theory and Examples* , Brooks/Cole, (2005).
- [Boucheron-Lugosi-Massart] S. Boucheron, G. Lugosi and P. Massart, *Concentration Inequalities* , Oxford, (2013).
- [Norris] Norris, J.R. , *Markov Chains* , Cambridge, (1997).

Universidade Federal da Bahia-UFBA
Instituto de Matemática / Colegiado da Pós-Graduação em Matemática

Av. Adhemar de Barros, s/n, Campus de Ondina, Salvador-BA
CEP: 40170 -110
www.pgmat.ufba.br